

Contents

□ Prefix Sum & Difference Array — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	4
SECTION 6 — Time & Space Complexity	5
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	6
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	7
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	8
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	9
SECTION 15 — Common Mistakes	12
SECTION 16 — Optimization Techniques	12
SECTION 17 — Multiple Language Templates	12
SECTION 18 — Dry Runs	14
SECTION 19 — Advanced Concepts	14
SECTION 20 — Senior Engineer Insights	14
SECTION 21 — Cheat Sheet	15
SECTION 22 — Revision Notes (5-Minute Review)	15
SECTION 23 — Practice Roadmap	15
SECTION 24 — Memory Tricks	16
SECTION 25 — Final Summary	16

□ Prefix Sum & Difference Array — Complete Interview Handbook

Pattern #5 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Arrays / Prefix Sum section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Prefix Sum precomputes a running cumulative aggregate $P[i] = A[0] + A[1] + \dots + A[i-1]$ so that the sum of **any** range $[l, r]$ can be answered in $O(1)$ via $P[r+1] - P[l]$. The **Difference Array** is its dual: precompute $D[i] = A[i] - A[i-1]$ so that **range updates** (add a value to every element in $[l, r]$) can be applied in $O(1)$, with the actual array reconstructed via a prefix sum over D at the end.

1.2 Why Was This Pattern Invented?

Naively answering Q range-sum queries over an array of size n costs $O(n)$ per query ($O(nQ)$ total). If the array doesn't change between queries, this is wasteful — the same sub-sums get recomputed repeatedly. Prefix Sum trades a one-time $O(n)$ preprocessing cost for $O(1)$ per query, exactly analogous to memoization. The Difference Array applies the same idea in reverse for updates: rather than updating every element in a range ($O(\text{range length})$ per update), you make two $O(1)$ edits and defer the "spreading" of the effect to a single final $O(n)$ reconstruction pass.

1.3 Real Intuition Behind The Pattern

Think of a **running bank balance ledger**. Instead of re-adding every deposit and withdrawal from day 1 to compute your balance on day 500, you keep a running total updated daily — checking the balance on any day is then just reading one number. Prefix Sum is exactly this running total; querying a range is "balance at end" minus "balance at start."

1.4 Mental Model

Prefix Sum turns "sum of a range" into "difference of two single values." Difference Array turns "update a range" into "mark the boundary of the effect, twice." Both exploit the fact that **cumulative effects can be represented compactly as boundary markers** rather than exhaustively per-element operations.

1.5 Visual Explanation

Array: [2, 4, 6, 8, 10]
Prefix: [0, 2, 6, 12, 20, 30] ($P[0]=0, P[i]=P[i-1]+A[i-1]$)

Sum of range $[1,3]$ (0-indexed, inclusive) = $P[4] - P[1] = 20 - 2 = 18$
Check: $4+6+8 = 18$

Difference Array update: add 5 to range $[1,3]$ of array of size 5

$D = [0,0,0,0,0,0]$ (size $n+1$)

$D[1] += 5 \rightarrow D[1]=5$

$D[4] -= 5 \rightarrow D[4]=-5$

Reconstruct via prefix sum of D : $[0,5,5,5,0] \rightarrow$ matches "+5 applied to indices 1..3 only"

1.6 Simple Analogy

Prefix Sum is like a **odometer reading** in a car — to find distance traveled between two points on a trip, you don't re-measure the road; you just subtract the earlier odometer reading from the later one. Difference Array is like **marking "start construction here" and "end construction here" signs** on a highway instead of repainting every mile of road individually.

1.7 When Should I Immediately Think About Using This Pattern?

- Multiple **range-sum queries** on a **static** (unchanging) array.

- “Subarray sum equals K” style problems (combined with hashing).
- Multiple **range-update** operations followed by a single final read of the array.
- 2D grid range-sum queries (2D prefix sum / summed-area table).
- Problems mentioning “**number of subarrays with sum...**”

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“range sum query”	Direct Prefix Sum signal
“subarray sum equals k”	Prefix Sum + hashmap of prefix sum frequencies
“multiple queries” over a static array	Precompute prefix sums once
“apply k updates to ranges, then return final array”	Difference Array signal
“number of subarrays” with a sum/count condition	Prefix Sum + hashmap counting

2.2 Hidden Hints

Constraints like “up to 10^5 queries on an array of up to 10^5 elements” scream $O(1)$ per query — ruling out recomputation and confirming Prefix Sum preprocessing is expected.

2.3 Interview Clues

Interviewer says “this array won’t change” or “you’ll be asked this many times” — a static-data signal for precomputation.

2.4 Common Trick Words

“Cumulative,” “running total,” “how many times does the sum equal,” “range increment” — all map directly to Prefix Sum / Difference Array vocabulary.

2.5 What Interviewers Expect

Correct 0-indexed vs 1-indexed boundary handling ($P[r+1] - P[l]$, not $P[r] - P[l]$), and recognizing when to combine Prefix Sum with a hashmap for “count subarrays with sum K” (rather than checking every pair of prefix indices in $O(n^2)$).

2.6 When NOT To Use This Pattern

- The array is **frequently updated** (single-element updates) **and** frequently queried — Prefix Sum requires $O(n)$ to rebuild after every update, which is too slow if both operations are frequent; use a **Fenwick Tree / Segment Tree** instead (Pattern #31 territory, advanced).
- The aggregate isn’t associative/invertible in a simple way (e.g., you need range **maximum**, not sum — max has no simple “subtract” inverse) — Prefix Sum doesn’t directly generalize to min/max range queries; use Sparse Tables or Segment Trees instead.

SECTION 3 — Decision Framework

Is the array STATIC (no updates) with MULTIPLE range-sum queries?

Yes → USE PREFIX SUM ($O(n)$ preprocess, $O(1)$ per query)

No

Do you have MULTIPLE RANGE UPDATES followed by ONE final read?

Yes → USE DIFFERENCE ARRAY ($O(1)$ per update, $O(n)$ final reconstruction)

No

Do you have BOTH frequent updates AND frequent range queries interleaved?

Yes → USE FENWICK TREE / SEGMENT TREE instead (Prefix Sum alone is too slow to rebuild each time)

No

Is the aggregate MIN/MAX (not sum) over ranges?

Yes → USE SPARSE TABLE (static, $O(1)$ query) or SEGMENT TREE (dynamic) instead

Why: Prefix Sum's entire value proposition rests on the array being static between the preprocessing step and the queries; the moment updates and queries interleave frequently, the $O(n)$ rebuild cost per update destroys the benefit, and you need a data structure supporting both operations in $O(\log n)$.

SECTION 4 — Why This Pattern Works

Mathematical: By definition, $P[i] = \sum_{k=0 \text{ to } i-1} A[k]$. The sum of range $[l, r]$ (inclusive) is $\sum_{k=l \text{ to } r} A[k] = P[r+1] - P[l]$, a direct consequence of telescoping sums — all terms before index l cancel out. This is provably correct by simple algebraic subtraction, no approximation involved.

Logical: Precomputing P once costs $O(n)$ (one pass, one addition per element). Each subsequent query costs $O(1)$ (one subtraction). Total cost for Q queries: $O(n + Q)$ instead of $O(n \cdot Q)$ — a massive improvement when Q is large.

Intuitive: “Total up to here, minus total up to there, equals total in between” — the same logic as bank balances or odometer readings.

Difference Array correctness proof: Applying $D[l] += \text{val}$; $D[r+1] -= \text{val}$ and then taking the prefix sum of D reconstructs an array where every index in $[l, r]$ has val added and every index outside does not, because the prefix sum of D **accumulates** the $+\text{val}$ starting at l and the $-\text{val}$ at $r+1$ **cancels** that accumulation exactly at the point the range ends — again a telescoping argument, symmetric to the Prefix Sum proof.

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Prefix Sum)

1. Build P of size $n+1$, $P[0] = 0$.
2. For i from 0 to $n-1$: $P[i+1] = P[i] + A[i]$.

3. For any query $[l, r]$ (0-indexed, inclusive): $\text{answer} = P[r+1] - P[l]$.

5.2 Step-by-Step Framework (Difference Array)

1. Build D of size $n+1$, all zeros.
2. For each update (l, r, val) : $D[l] += \text{val}$; $D[r+1] -= \text{val}$.
3. After all updates, reconstruct: $A[i] = A[i] + \text{prefixSum}(D)[i]$ (running sum of D up to index i).

5.3 General Template — Prefix Sum

```
function buildPrefixSum(array):
    n = length(array)
    P = new array of size n+1, P[0] = 0
    for i in range(0, n):
        P[i+1] = P[i] + array[i]
    return P

function rangeSum(P, l, r):      # inclusive [l, r], 0-indexed
    return P[r+1] - P[l]
```

5.4 General Template — Difference Array

```
function applyRangeUpdate(D, l, r, val):
    D[l] += val
    D[r+1] -= val

function reconstruct(original, D):
    n = length(original)
    result = new array of size n
    running = 0
    for i in range(0, n):
        running = running + D[i]
        result[i] = original[i] + running
    return result
```

5.5 Interview Thinking Process

1. “There are multiple range-sum queries on a static array — I’ll precompute a prefix sum array in $O(n)$ so each query is $O(1)$.”
2. “For range sum $[l, r]$, I use $P[r+1] - P[l]$ — careful with the off-by-one at the boundary.”
3. “If instead I have multiple range *updates* and only need the final array, I’ll use a difference array — $O(1)$ per update, one final $O(n)$ reconstruction.”
4. “If updates and queries are interleaved and frequent, Prefix Sum alone isn’t enough — I’d need a Fenwick Tree or Segment Tree for $O(\log n)$ both ways.”

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n)$ preprocess + $O(1)$ per query (Prefix Sum)	$O(n)$	One pass to build, then $O(1)$ lookups

Case	Time	Space	Why
Worst Case (Difference Array)	$O(1)$ per update + $O(n)$ final reconstruction	$O(n)$	Updates deferred, applied once at the end
Average Case	Same as worst — no data dependency	$O(n)$	Deterministic preprocessing cost
Best Case	$O(n)$ minimum (must scan once even for a single query)	$O(n)$	Preprocessing is unavoidable overhead
Amortized (Q queries)	$O((n + Q) / Q)$ per query $\approx O(1)$ for large Q	$O(n)$	Preprocessing cost amortizes across many queries

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty array	<code>[]</code>	Prefix sum array is just <code>[0]</code> ; no valid ranges
Single element	<code>[5]</code>	$P = [0, 5]$; range $[0,0] = P[1]-P[0] = 5$
Query range out of bounds	$l > r$ or negative indices	Validate/guard explicitly, define behavior (often invalid input)
Negative numbers in array	<code>[-2, 3, -1]</code>	Works identically — prefix sum handles negatives without special-casing
Full-array range query	<code>[0, n-1]</code>	$P[n] - P[0] = \text{total array sum}$, sanity-check this
Difference array range covering entire array	<code>[0, n-1]</code>	$D[n]$ must exist (size $n+1$) to safely apply the $-val$ at the boundary
Overlapping range updates	Two updates both touching index 5	Difference array correctly accumulates both via superposition — no special handling needed
2D prefix sum with single row/column	Degenerate grid	Formula reduces cleanly to 1D case: $S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + A[i][j]$

Common mistakes: off-by-one using $P[r] - P[l]$ instead of $P[r+1] - P[l]$; forgetting the difference array needs size $n+1$ (not n) to safely mark the $r+1$ boundary without an out-of-bounds error when $r = n-1$.

SECTION 8 — Pros & Cons

Advantages: $O(1)$ range-sum queries after $O(n)$ preprocessing; $O(1)$ range updates with Difference Array; simple, purely arithmetic, no complex data structures needed. **Disadvantages:**

Static-data assumption — any single-element update invalidates the entire prefix sum array, requiring an $O(n)$ rebuild; doesn't generalize to min/max range queries. **Trade-offs:** Prefix Sum ($O(n)$ space, $O(1)$ query, no updates) vs. Fenwick/Segment Tree ($O(n)$ space, $O(\log n)$ query AND update) — choose Prefix Sum only when updates are absent or batched at the very end. **Limitations:** Sum-only (or other invertible/associative aggregates like XOR); not usable for min/max/gcd ranges directly. **Inefficient when:** frequent interleaved updates + queries — rebuilding $O(n)$ per update against many updates degrades to $O(nQ)$, no better than brute force.

SECTION 9 — Real World Applications

Domain	Application
Google	Search analytics dashboards computing rolling/range aggregates over static historical logs
Amazon	Inventory range queries — “total stock across warehouses 10-50” computed via precomputed cumulative sums
Meta	Ad impression range analytics over fixed time-bucketed static datasets
Netflix	Precomputed cumulative watch-time histograms for range-based analytics dashboards
Banking/Payments	Running account balance ledgers — exactly the prefix sum concept applied to transaction logs
Databases	Materialized cumulative aggregate columns (window functions like <code>SUM() OVER</code>) implemented via prefix-sum-like logic
Image Processing	2D prefix sum (“summed-area table”) for $O(1)$ box-filter/blur computation over rectangular regions
Operating Systems	Resource usage accounting over time windows via cumulative counters
Networking	Bandwidth usage range queries over fixed time-bucketed traffic logs
AI/ML	Efficient computation of rolling statistics (cumulative sums) for feature engineering over time-series data

SECTION 10 — Interview Strategy

How seniors answer: They immediately identify whether the problem is query-heavy (Prefix Sum) or update-heavy-then-read (Difference Array), state the $O(n)$ preprocessing / $O(1)$ query trade-off explicitly, and proactively raise the Fenwick/Segment Tree alternative if updates and queries interleave.

How juniors answer: They often recompute range sums from scratch per query ($O(n)$ per query) without recognizing the precomputation opportunity, or they get the $P[r+1] - P[l]$ boundary wrong under pressure.

Typical follow-ups: “What if the array can be updated between queries?” (Fenwick/Segment Tree). “Can you extend this to 2D?” (2D prefix sum / summed-area table). “How would you count subarrays with sum exactly K?” (Prefix Sum + hashmap of running sum frequencies).

Optimization questions: “Can you avoid $O(n)$ extra space for the prefix array?” (Only if queries can be answered in a single pass without needing arbitrary range access later — usually not avoidable if queries come after all data is seen).

SECTION 11 — Pattern Variations

Variation	Description	Example
1D Prefix Sum	Range sum queries on a 1D array	Range Sum Query - Immutable
2D Prefix Sum (Summed-Area Table)	Range sum queries on a 2D grid	Range Sum Query 2D - Immutable
Prefix Sum + Hashmap	Count/find subarrays with a given sum	Subarray Sum Equals K
Prefix XOR	Same idea using XOR instead of sum (XOR is its own inverse)	XOR Queries of a Subarray
Difference Array (1D)	Efficient range updates	Corporate Flight Bookings
Difference Array (2D)	Efficient rectangle-range updates	Range Addition II family
Prefix Sum + Binary Search	Find boundary index where cumulative sum crosses a threshold	Capacity-related search problems

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Sliding Window	Handles contiguous ranges incrementally during a single scan, doesn't support arbitrary random-access range queries after the fact	Need running window state during a single pass, not stored for later arbitrary queries
Fenwick Tree / Segment Tree	Supports both updates and queries in $O(\log n)$	Frequent interleaved updates and queries
Two Pointers	Solves pair/subrange search on sorted/monotonic data, not general range-sum precomputation	Pair/triplet search, not arbitrary range-sum retrieval

Comparison Table

Aspect	Prefix Sum	Fenwick Tree	Sliding Window
Query time	$O(1)$	$O(\log n)$	N/A (single pass only)

Aspect	Prefix Sum	Fenwick Tree	Sliding Window
Update time	O(n) rebuild	O(log n)	N/A
Space	O(n)	O(n)	O(k)
Best for	Static data, many queries	Dynamic data, many updates+queries	Single-pass contiguous range problems

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct 1D range sum queries	Range Sum Query - Immutable, Running Sum of 1d Array
Medium	Prefix Sum + hashmap combination	Subarray Sum Equals K, Continuous Subarray Sum
Hard	2D prefix sum, difference arrays with multiple updates	Range Sum Query 2D, Corporate Flight Bookings
Very Hard	Combined with binary search or advanced counting	Count of Range Sum, Maximum Size Subarray Sum Equals K variants at scale

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Range Sum Query - Immutable	Easy	Amazon, Google	Direct 1D prefix sum	Foundational prefix sum
2	Running Sum of 1d Array	Easy	Amazon	Direct prefix sum construction	Basic mechanics
3	Range Sum Query 2D - Immutable	Medium	Amazon, Google, Meta	2D prefix sum / summed area table	Dimensional extension
4	Subarray Sum Equals K	Medium	Amazon, Meta, Google, Microsoft	Prefix sum + hashmap frequency counting	Core hashmap+prefix combination
5	Continuous Subarray Sum	Medium	Meta, Amazon	Prefix sum + modulo hashing	Modular arithmetic + prefix sum
6	Subarray Sums Divisible by K	Medium	Google, Amazon	Prefix sum + modulo bucket counting	Modulo counting variant

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
7	Corporate Flight Bookings	Medium	Amazon, Google	Classic difference array range update	Difference array mechanics
8	Range Addition	Medium	Google	Difference array with final reconstruction	Difference array mechanics
9	Product of Array Except Self	Medium	Amazon, Meta, Microsoft, Apple	Prefix/suffix product (prefix sum analog)	Prefix product variant
10	Maximum Size Subarray Sum Equals K	Medium	Meta, Google	Prefix sum + hashmap of first occurrence	Length-tracking prefix sum variant
11	Contiguous Array	Medium	Amazon, Meta	Prefix sum with transformed values (0→-1)	Transformation trick
12	Count Number of Nice Subarrays	Medium	Google	Prefix sum on transformed parity array	Transformation + counting
13	Binary Subarrays With Sum	Medium	Google, Amazon	Prefix sum + hashmap counting	Exact-sum counting
14	XOR Queries of a Subarray	Medium	Google, Amazon	Prefix XOR (self-inverse aggregate)	XOR variant of prefix sum
15	Find Pivot Index	Easy	Amazon, Google	Prefix sum comparison from both sides	Two-sided prefix sum comparison
16	Range Sum Query - Mutable	Medium	Amazon, Google	Contrast: requires Fenwick Tree, not pure prefix sum	Recognizing pattern limits
17	Count of Range Sum	Hard	Google, Amazon	Prefix sum + merge sort / BIT for counting	Advanced counting with sorted structures
18	Path Sum III	Medium	Amazon, Meta	Prefix sum applied along tree root-to-node paths	Cross-pattern (tree + prefix sum)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
19	Number of Subarrays with Bounded Maximum	Medium	Google	Related counting technique (not pure prefix sum but adjacent)	Boundary-based counting
20	Minimum Operations to Reduce X to Zero	Medium	Amazon	Prefix+suffix sum combination	Two-sided prefix sum
21	Range Addition II	Medium	Google	2D difference array style reasoning	2D update reasoning
22	Car Pooling	Medium	Amazon, Google	Difference array on a timeline of pickups/drop-offs	Timeline difference array
23	My Calendar III	Hard	Google, Amazon	Difference array / sweep line combination	Advanced timeline aggregation
24	Grid Illumination	Hard	Google	2D prefix-sum-adjacent counting with hashmaps	Grid-based counting
25	Sum of All Subset XOR Totals (contrast)	Easy	Amazon	Contrast: combinatorics, not prefix sum	Pattern-boundary awareness
26	K Radius Subarray Averages	Medium	Amazon	Fixed window sum via prefix sum	Fixed window + prefix sum combination
27	Maximum Subarray Sum After One Operation	Medium	Google	Prefix/suffix combination with modification	Advanced prefix+suffix reasoning
28	Number of Ways to Split Array	Medium	Amazon	Prefix sum for split-point validity checking	Split-point counting
29	Matrix Block Sum	Medium	Google, Amazon	2D prefix sum with clamped boundaries	2D boundary handling

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
30	Booking Concert Tickets (Range Update variant)	Medium	Amazon (custom/interview variant)	Difference array for seat range booking	Difference array real-world framing

SECTION 15 — Common Mistakes

1. Using $P[r] - P[l]$ instead of $P[r+1] - P[l]$ for inclusive range $[l, r]$. *Fix:* always define $P[0] = 0$ and treat $P[i]$ as “sum of first i elements,” never conflate with $A[i]$.
2. Difference array sized n instead of $n+1$, causing an out-of-bounds write when $r = n-1$. *Fix:* always allocate size $n+1$.
3. Forgetting to reset/rebuild the prefix sum array after any update to the underlying array — leads to stale, incorrect query results. *Fix:* if updates happen, either rebuild explicitly or switch to Fenwick/Segment Tree.
4. Applying Prefix Sum to min/max range queries, which don’t have a valid “subtract” inverse. *Fix:* recognize non-invertible aggregates and use Sparse Table/Segment Tree instead.
5. In “subarray sum equals K ” problems, forgetting to initialize the hashmap with $\{0: 1\}$ (representing the empty prefix) — causes undercounting of subarrays starting at index 0. *Fix:* always seed the frequency map with the base case.

Why people fail: the arithmetic is simple, but the off-by-one boundary handling and the need to seed hashmap base cases are exactly the kind of subtle details that get skipped under time pressure, leading to answers that are “almost right” but fail edge cases in judges/interviews.

SECTION 16 — Optimization Techniques

- **Time:** Precompute once, reuse across all queries — never recompute per query.
- **Space:** For 2D problems, in-place prefix sum computation (overwriting the original grid) can save space if the original grid isn’t needed afterward.
- **Readability:** Clearly comment the indexing convention ($P[i]$ = sum of first i elements) since off-by-one confusion is the #1 bug source.
- **Interview performance:** State the indexing convention explicitly before coding (e.g., “I’ll use a 1-indexed prefix array where $P[0]=0$ ”) to preempt confusion and demonstrate rigor.

SECTION 17 — Multiple Language Templates

Java

```
class PrefixSum {
    int[] prefix;
    PrefixSum(int[] arr) {
        prefix = new int[arr.length + 1];
        for (int i = 0; i < arr.length; i++) prefix[i+1] = prefix[i] + arr[i];
    }
    int rangeSum(int l, int r) { return prefix[r+1] - prefix[l]; }
}
```

JavaScript

```
function buildPrefixSum(arr) {
    const prefix = new Array(arr.length + 1).fill(0);
    for (let i = 0; i < arr.length; i++) prefix[i+1] = prefix[i] + arr[i];
    return prefix;
}
function rangeSum(prefix, l, r) { return prefix[r+1] - prefix[l]; }
```

PHP

```
function buildPrefixSum(array $arr): array {
    $n = count($arr);
    $prefix = array_fill(0, $n + 1, 0);
    for ($i = 0; $i < $n; $i++) $prefix[$i+1] = $prefix[$i] + $arr[$i];
    return $prefix;
}
function rangeSum(array $prefix, int $l, int $r): int {
    return $prefix[$r+1] - $prefix[$l];
}
```

Python

```
def build_prefix_sum(arr):
    prefix = [0] * (len(arr) + 1)
    for i, v in enumerate(arr):
        prefix[i+1] = prefix[i] + v
    return prefix

def range_sum(prefix, l, r):
    return prefix[r+1] - prefix[l]
```

Go

```
func buildPrefixSum(arr []int) []int {
    prefix := make([]int, len(arr)+1)
    for i, v := range arr {
        prefix[i+1] = prefix[i] + v
    }
    return prefix
}
func rangeSum(prefix []int, l, r int) int {
    return prefix[r+1] - prefix[l]
}
```

C++

```
vector<int> buildPrefixSum(vector<int>& arr) {
    vector<int> prefix(arr.size() + 1, 0);
    for (int i = 0; i < (int)arr.size(); i++) prefix[i+1] = prefix[i] + arr[i];
    return prefix;
}
int rangeSum(vector<int>& prefix, int l, int r) { return prefix[r+1] - prefix[l]; }
```

SECTION 18 — Dry Runs

Small Input

arr = [3, 1, 4, 1, 5]

P = [0, 3, 4, 8, 9, 14]

Query [1,3] (elements 1,4,1): $P[4]-P[1] = 9-3 = 6$ (1+4+1=6)

Query [0,4] (whole array): $P[5]-P[0] = 14-0 = 14$ (3+1+4+1+5=14)

Large Input (Conceptual)

For an array of 10^6 elements with 10^5 queries, preprocessing costs 10^6 operations once; each of the 10^5 queries costs $O(1)$ — total $\sim 1.1M$ operations instead of a brute-force $10^5 \times 10^6 = 10^{11}$ operations, a difference of five orders of magnitude.

Corner Case

arr = [5], query [0,0]: P = [0,5], answer = $P[1]-P[0] = 5$. Difference array update covering the whole array [0, n-1] with val=10 on arr of size 5: $D[0]+=10$, $D[5]-=10$ — requires D to have size 6, confirming the $n+1$ sizing requirement.

SECTION 19 — Advanced Concepts

- **Prefix Sum + Hashmap for “count subarrays with sum K”:** maintain a running prefix sum and a hashmap of {prefixSumValue: frequency} seen so far; for each new prefix sum S, the count of valid subarrays ending here is $\text{freq}[S - K]$ — this turns an $O(n^2)$ brute force into $O(n)$.
- **2D Prefix Sum (Summed-Area Table):** $S[i][j] = A[i][j] + S[i-1][j] + S[i][j-1] - S[i-1][j-1]$ (inclusion-exclusion to avoid double-counting the overlapping region) — a direct generalization of the 1D telescoping idea.
- **Prefix XOR:** since XOR is its own inverse ($a \oplus a = 0$), range-XOR queries use $P[r+1] \oplus P[l]$ exactly like sum, just with XOR instead of addition/subtraction — recognizing which operations are “invertible” this way (sum, XOR, but not min/max) is a key generalization insight.
- **Difference Array in 2D:** for rectangle-range updates, apply four corner markers (+val at top-left, -val at top-right+1 and bottom-left+1, +val at bottom-right+1) then take a 2D prefix sum to reconstruct — a direct extension of the inclusion-exclusion principle.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Prefix Sum as the simplest instance of a much larger idea: **trading write-time cost for read-time cost via precomputed cumulative structures** — the same principle underlies materialized views in databases, CDN edge caching of aggregate metrics, and rolling window analytics dashboards. The interview signal they look for is whether a candidate can identify **which aggregate operations are “invertible”** (sum, XOR, product-with-no-zeros) versus which are not (min, max, mode) — and pivot to a Sparse Table or Segment Tree the moment the aggregate isn’t invertible, rather than trying to force Prefix Sum where it structurally cannot work.

SECTION 21 — Cheat Sheet

PATTERN: Prefix Sum / Difference Array

RECOGNIZE: multiple range-sum queries on static data (Prefix Sum) OR multiple range updates + one final range query

TEMPLATE (Prefix Sum):

```
P[0] = 0; P[i+1] = P[i] + A[i]
```

```
rangeSum(l, r) = P[r+1] - P[l]
```

TEMPLATE (Difference Array):

```
D[l] += val; D[r+1] -= val
```

```
reconstruct via prefix sum of D
```

COMPLEXITY: $O(n)$ preprocess, $O(1)$ per query/update

KEY PROOF: telescoping sum cancels all terms outside the target range

WATCH FOR: off-by-one ($P[r+1]$ not $P[r]$), array sizing ($n+1$), non-invertible aggregates (min/max)

DOESN'T APPLY WHEN: frequent interleaved updates+queries (use Fenwick/Segment Tree), min/max ranges (use Segment Tree)

SECTION 22 — Revision Notes (5-Minute Review)

- Prefix Sum: $P[i]$ = sum of first i elements; range $[l, r]$ sum = $P[r+1] - P[l]$.
- Difference Array: mark $+val$ at l , $-val$ at $r+1$; reconstruct via prefix sum of the diff array.
- Both rely on telescoping/cancellation — only works for invertible aggregates (sum, XOR), not min/max.
- Combine with hashmap for “count subarrays with sum K ” — seed map with $\{0:1\}$ for the empty prefix.
- Not suitable when updates and queries interleave frequently — use Fenwick/Segment Tree there.
- 2D generalization uses inclusion-exclusion (summed-area table).

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	1D prefix sum mechanics	Range Sum Query - Immutable (303), Running Sum of 1d Array (1480), Find Pivot Index (724)
Intermediate	Prefix sum + hashmap combination	Subarray Sum Equals K (560), Contiguous Array (525), Continuous Subarray Sum (523)
Advanced	2D prefix sum, difference arrays	Range Sum Query 2D - Immutable (304), Corporate Flight Bookings (1109), Car Pooling (1094)
Expert	Advanced counting, cross-pattern	Count of Range Sum (327), Path Sum III (437), My Calendar III (732)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Running total in, subtract out” — build a running total, subtract to get any range.
 - **Visualization:** An **odometer** — distance between two points = later reading minus earlier reading, never re-measure the road.
 - **Recognition shortcut:** “Many range-sum queries, static array” → Prefix Sum. “Many range updates, one final read” → Difference Array.
-

SECTION 25 — Final Summary

Prefix Sum and its dual, the Difference Array, exploit the **telescoping property of invertible aggregates** to convert $O(n)$ per-query or per-update costs into $O(1)$, at the price of $O(n)$ one-time preprocessing and a static-data assumption. The single most important thing to remember forever: **$P[r+1] - P[l]$ gives the sum of range $[l, r]$, and this only works because addition (and XOR) has a clean inverse — the moment you need min/max ranges or frequent interleaved updates, this pattern’s assumptions break and you must reach for Sparse Tables, Fenwick Trees, or Segment Trees instead.**